Module: quiz and requests

English version of ../../modules/quiz.md.

Files: app/routers/quiz.py, app/models.py (QuizSubmissionRequest), app/mailer.py.

1. Purpose and boundaries

Accept a quiz request, validate contacts and room parameters, store it, let the administrator list and moderate it. It does not send e-mail unless SMTP is configured (the notification is optional).

2. Use scenarios

- Filling in the quiz; consent to data processing is mandatory.
- Keeping the quiz state between steps (/api/session/save).
- Seeing your own requests; the admin sees all requests and changes status.
- Rejections: bad e-mail or phone → 400; no consent → 400.

3. Contract

POST /api/quiz/submit, GET /api/quiz/submissions, GET|PUT /api/admin/submissions. Request schema: QuizSubmissionRequest.

4. Data

Writes quiz_submissions (contacts, room parameters, status new by default).

5. Access rules

Submission is open; the list of all requests and the status change require the admin role checked on the server, not only in the UI (security.require_admin).

6. Dependencies

Outgoing: SQLite, mailer (optional). Incoming: static/quiz.js.

7. Configuration

SMTP_* (enable e-mail), DB_PATH.

8. Build and debug

```
python -m pytest tests/test_quiz.py -q
```

9. Tests

tests/test_quiz.py: a valid request, mandatory consent, e-mail validation, phone validation, the admin list is closed without the admin role.

10. Observability

The request flow is visible in the table and in the admin panel; errors go to stdout. Codes: 400 (validation), 401/403 (access).

11. Limitations and readiness

- No messenger notifications: only e-mail and the admin panel.
- No export of requests to a file.
- Readiness: validation is covered by tests, admin data is unreachable without the role.